@eshaaldev

Day 1

CSS Fundamentals

What is CSS?

CSS stands for Cascading Style Sheets

It's the language used to style HTML pages

Controls the look, feel, and layout of web content

Invented by Håkon Wium Lie in 1994 and first published in 1996

@eshaaldev

Syntax

```
/* Syntax */
selector {
    property: value;
  }
  /* Example */
 h1{
    color: purple;
 }
```

@eshaaldev

Ways to Write CSS

1. Inline

```
<h1 style="color: purple;"></h1>
```

2. Internal

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <title>CSS</title>
5  </head>
6  <style>
7      h1{
8          color: purple;
9      }
10 </style>
11 <body>
12     <h1 >Hello</h1>
13 </body>
14 </html>
```

@eshaaldev

Ways to Write CSS

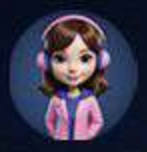
3. External

```
/* in a file called styles.css */

h1{
    color: purple;
 }
```

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>CSS</title>
    Link css to html
    <link rel="stylesheet" href="styles.css">
</head>

```



@eshaaldev

Selectors in CSS

the HTML elements you want to style.

Selector Type	Example	What it Selects
Element	`p`	All `<p>` tags
Class	`.title`	All elements with class "title"
ID	`#hero`	Element with id "hero"
Group	`h1, p`	All `<h1>` and `<p>`

@eshaaldev

Advance Selectors

Selector	Type	Description	Example
`*`	Universal	Targets **all elements**	`* { margin: 0; }`
`div > p`	Child selector	Targets **direct children** only	`div > p { color: red; }`
`ul li`	Descendant selector	Targets **all nested elements**	`ul li { list-style: none; }`
`h1 + p`	Adjacent sibling	Targets **next sibling** only	`h1 + p { margin-top: 0; }`
`h1 ~ p`	General sibling	Targets **all following siblings**	`h1 ~ p { color: gray; }`
`a:hover`	Pseudo-class	Targets when user **hovers** on element	`a:hover { color: green; }`
`li:first-child`	Pseudo-class	Targets the **first child** of parent	`li:first-child { font-weight: bold; }`
`input:checked`	Form pseudo-class	Targets **selected/checked** inputs	`input:checked { display: none; }`
`:not(.active)`	Negation selector	Targets all elements **not matching** rule	`button:not(.active)`
`[type="checkbox"]`	Attribute selector	Targets elements with exact **attribute value**	`input[type="checkbox"]`
`[href^="https"]`	Attribute selector	Targets attributes that **start with** value	`a[href^="https"]`
`p::before` / `::after`	Pseudo-elements	Insert content **before/after** element	`p::after { content: "."; }`

What is the Cascade?

It basically helps in deciding when multiple rules apply, which one wins.

CSS uses:

Specificity

Source Order

Importance (!important)

@eshaaldev

Specificity

More specific rules beat general ones

#id > .class > element

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <title>CSS</title>
    <link rel="stylesheet"
    href="styles.css">
</head>
<body>
        <h1 id="text"
        class="text">Heading</h1>
</body>
</html>
```

```css
#text {
    color: blue;
}
.text {
    color: sandybrown;
}
h1 {
    color: red;
}
```

Heading

@eshaaldev

!Important

Force a style to win
Use it carefully as it can create debugging issues.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <title>CSS</title>
    <link rel="stylesheet"
    href="styles.css">
</head>
<body>
        <h1 id="text"
        class="text">Heading</h1>
</body>
</html>
```

```css
#text {
    color: blue;
}
.text {
    color: sandybrown;
}
h1   {
    color: red  !important;  }
```

Heading

Order of Styles

Last one wins

```
h1 {
    color: blue;
}
h1 {
    color: sandybrown;
}
```

Heading